In order to maintain the accuracy of each member in the ensemble, the maximal prediction returned by each member should be consistent with the true label. Therefore, in our method, we define the ensemble diversity with respect to the *non-maximal* predictions. For a single input pair (x, y) in the training set, the maximal prediction should be consistent with the true label y, and the non-maximal predictions correspond to other labels except for y. Inspired by the theory of the determinant point process (DPP) (Kulesza et al., 2012), we define the ensemble diversity as

$$\mathbb{ED} = \det(\tilde{M}_{\setminus y}^{\top} \tilde{M}_{\setminus y}). \tag{3}$$

Here $\tilde{M}_{\setminus y} = (\tilde{F}_{\setminus y}^{1}, \cdots, \tilde{F}_{\setminus y}^{K}) \in \mathbb{R}^{(L-1)\times K}$, each column vector $\tilde{F}_{\setminus y}^{k} \in \mathbb{R}^{L-1}$ is obtained by normalizing $F_{\setminus y}^{k}$ under L_2-norm, where $F_{\setminus y}^{k}$ is the order preserving prediction of the k-th classifier on x without the y-th element. According to the matrix theory (Bernstein, 2005), we have:

$$\det(\tilde{M}_{\setminus y}^{\top} \tilde{M}_{\setminus y}) = \mathrm{Vol}^2(\{\tilde{F}_{\setminus y}^{k}\}_{k\in[K]}), \tag{4}$$

where $\mathrm{Vol}(\cdot)$ denotes the volume spanned by the vectors of the input set. This provides an elegant geometric interpretation for the ensemble diversity, as intuitively shown in Fig. 1. Since $\tilde{F}_{\setminus y}^{k}$ is normalized, i.e., $\|\tilde{F}_{\setminus y}^{k}\|_2 = 1$, the ensemble diversity $\mathbb{ED}$ achieves its maximal value 1 if and only if the column vectors of $\tilde{M}_{\setminus y}$ are mutually orthogonal.

Note that most of the previous definitions of diversity, e.g., those defined on prediction errors, include the predictions F_y^k on the true label y (Liu & Yao, 1999a;b; Islam et al., 2003; Kuncheva & Whitaker, 2003). Since DNNs are no longer weak classifiers in the most cases (Goodfellow et al., 2016), encouraging F_y^k to be diverse may make the loss on accuracy outweighs the gain for DNNs, or render the convergence point of the training process uncontrollable. In contrast, we define the diversity among the normalized non-maximal predictions $\tilde{F}_{\setminus y}^{k}$, which allows the maximal prediction of each network to be consistent with the true label, and thus will not affect ensemble accuracy. What's more, promoting this diversity could improve robustness. Because, in the adversarial setting, when the maximal prediction corresponds to the true label y, the non-maximal predictions correspond to the labels $[L]\setminus\{y\}$, which include all potentially wrong labels returned for the adversarial examples. Thus a high diversity or inconsistency on the non-maximal predictions $F_{\setminus y}^{k}$ can lower down the transferability of adversarial examples among the networks, and further lead to better robustness of the ensemble.

To promote ensemble diversity, we propose the **adaptive diversity promoting (ADP)** regularizer as

$$\mathrm{ADP}_{\alpha,\beta}(x, y) = \alpha \cdot \mathcal{H}(\mathcal{F}) + \beta \cdot \log(\mathbb{ED}) \tag{5}$$

for a single input pair (x, y), where $\alpha, \beta \geq 0$ are two hyperparameters. Note that the ADP regularizer consists of two parts. The first part is the ensemble Shannon entropy $\mathcal{H}(\mathcal{F})$, and the second part is the logarithm of the ensemble diversity (LED), which we call as the LED part. More technical details about the effects of these two parts and why the ensemble entropy part is necessary will be deferred to Section 3.3. The training procedure with the ADP regularizer is called the ADP training method, as described in Algorithm 1. Further analysis on other potential definition of ensemble diversity is provided in Appendix B.1.

Algorithm 1 The ADP training procedure

Input: The K individual models $\{F^k(x, \theta^k)\}_{k\in[K]}$; the training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i\in[N]}$.
Initialization: Initialize each θ^k as θ_0^k, the training step counters as $c_k = 0$, ε_k be the learning rate variables, $I = [K]$ be the indicator set, where $k \in [K]$.
while $I \neq \varnothing$ **do**
  Calculate the objective on a mini-batch of data $\mathcal{D}_m$
  $$\mathcal{L}_{\mathrm{ADP}}^{m} = \frac{1}{|\mathcal{D}_m|} \sum_{(x_i,y_i)\in\mathcal{D}_m} [\mathcal{L}_{\mathrm{ECE}} - \mathrm{ADP}_{\alpha,\beta}](x_i, y_i).$$
  for k' in I **do**
    Update $\theta_{c+1}^{k'} \leftarrow \theta_c^{k'} - \varepsilon_{k'} \nabla_{\theta^{k'}} \mathcal{L}_{\mathrm{ADP}}^{m}\big|_{\{\theta_{c_k}^{k}\}_{k\in[K]}}$.
    Update $c_{k'} \leftarrow c_{k'} + 1$, where $c = c_{k'}$.
    if $\theta^{k'}$ converges **then** Update $I = I\setminus\{k'\}$.
  end for
end while
Return: The parameters $\theta^k = \theta_{c_k}^k$, where $k \in [K]$.

When performing back-propagation (Rumelhart et al., 1986) in the ADP training procedure, we need to separately perform two matrix operations $\det(G)$ and G^{-1}. Without loss of generality, let $G \in \mathbb{R}^{m\times m}$, where m denote the dimension of the matrix, these two operations both have computational complexities of $\mathcal{O}(m^3)$ under common algorithms (Kulesza et al., 2012), which may cause computational obstacle if m scales comparably with the size of the problem. For example, in previous DPP-based methods used to sample diverse subsets (Kwok & Adams, 2012; Mariet & Sra, 2016), they need to calculate the values of $\det(G)$ to obtain the sampling probability for each subset, where m usually scales with the number of classes or number of hidden units in networks. However, in our method, we have $G = \tilde{M}_{\setminus y}^{\top} \tilde{M}_{\setminus y}$ and $m = K$, where K grows much slower with the scale of problem (Russakovsky et al., 2015). This property enables our method to scale to most of the modern machine learning tasks and avoid similar computational obstacle encountered in previous work. As experimentally shown in Section 4, our method is also compatible with other defenses acting on individual networks, e.g., adversarial training (Goodfellow et al., 2015; Madry et al., 2018), and thus our method could be an orthogonal approach to further improve robustness for ensemble models. These results verify that ensemble diversity is indeed an effective

component to consider when designing stronger defenses.

3.3. Theoretical Analyses on ADP Training

The minimization problem in the ADP training on a single input-label pair (x, y) can be formally denoted as

$$\begin{aligned} &\min_{\theta} \mathcal{L}_{\text{ECE}} - \text{ADP}_{\alpha,\beta} \\ &\text{s.t. } 0 \leq F_j^k \leq 1, \sum_{j\in[L]} F_j^k = 1, \end{aligned} \tag{6}$$

where $\theta = \{\theta^k\}_{k\in[K]}$ denotes all the trainable parameters in the ensemble. For multiple input-label pairs, the objective function is simply a summation of each single function as in Algorithm 1. However, to simplify the analysis, we focus on the optimal solution in the space of output predictions $\mathbb{F} = \{F^k\}_{k\in[K]}$, and assume that the representation capacity of each network is unlimited (Hornik et al., 1989). Therefore, we ignore the specific mapping from x to each F^k, and redefine the above minimization problem with respect to $\mathbb{F}$, rather than θ. Below we will theoretically analyze how the values of the two hyperparameters α and β influence the solution for the minimization problem. If $\alpha = 0$, the ADP regularizer degenerates to the LED part. In this case, the solution will be one-hot vectors 1_y just as there is no regularizer, as shown in the following theorem:

Theorem 1. *(Proof in Appendix A) If $\alpha = 0$, then $\forall \beta \geq 0$, the optimal solution of the minimization problem (6) satisfies the equations $F^k = 1_y$, where $k \in [K]$.*

The conclusion means that the LED part in the ADP regularizer cannot work alone without the ensemble entropy part, indicating that the ensemble entropy part is necessary. In a nutshell, this is because we define the LED part on the *normalized* predictions $\tilde{F}^k_{\backslash y}$ corresponding to untrue labels. Thus the LED part can only affect the mutual angles of these untrue predictions. The summation of all unnormalized untrue predictions is $1 - F_y^k$, and in this case, the summation will only be influenced by the ECE loss part, which leads to one-hot optimal solution 1_y. On the other hand, if $\beta = 0$, the ADP regularizer degenerates to the ensemble entropy, which results in similar optimal solutions on ensemble prediction $\mathcal{F}$ as label smoothing (Szegedy et al., 2016). The equivalent smoothing factor is $1 - \mathcal{F}_y$, and the conclusions are formally stated in the following theorem:

Theorem 2. *(Proof in Appendix A) When $\alpha > 0$ and $\beta = 0$, the optimal solution of the minimization problem (6) satisfies the equations $F_y^k = \mathcal{F}_y$, $\mathcal{F}_j = \frac{1-\mathcal{F}_y}{L-1}$ and*

$$\frac{1}{\mathcal{F}_y} = \frac{\alpha}{K} \log \frac{\mathcal{F}_y(L-1)}{1-\mathcal{F}_y}, \tag{7}$$

where $k \in [K]$ and $j \in [L]\backslash\{y\}$.

Here Eq. (7) can assist us in selecting proper values for the hyperparameter α. For example, if we want to train an ensemble of 5 individual networks ($K = 5$) on ImageNet ($L = 1000$) with equivalent label smoothing factor 0.1 ($\mathcal{F}_y = 0.9$), then we can calculate from Eq. (7) that $\alpha \approx 0.61$. The difference of the solutions in Theorem 2 from those of label smoothing is that there is an extra constraint for each individual prediction: $F_y^k = \mathcal{F}_y$. This guarantees the consistency of all maximal predictions with the true label. Besides, it is worthy to note that for any $j \in [L]\backslash\{y\}$, the ensemble entropy part does not directly regularize on the non-maximal predictions F_j^k, but regularize on their mean, i.e., the ensemble prediction $\mathcal{F}_j$. This leaves degrees of freedom on the optimal solutions of F_j^k, and the effect of the LED part is to further regularize on them: if $L - 1$ is divisible by K, i.e., $K \mid (L-1)$, the non-maximal predictions of each network are mutually orthogonal, as intuitively shown in the right panel of Fig. 1, where $K = 3$ and $L = 10$. We mathematically denote the optimal solution in this case as in the following corollary:

Corollary 1. *If there is $K \mid (L-1)$, then $\forall \alpha, \beta > 0$, the optimal solution of the minimization problem (6) satisfies the Eq. (7). Besides, let $S = \{s_1, , \cdots, s_K\}$ be any partition of the index set $[L]\backslash\{y\}$, where $\forall k \in [K]$, $|s_k| = \frac{L-1}{K}$. Then the optimal solution further satisfies:*

$$F_j^k = \begin{cases} \frac{K(1-\mathcal{F}_y)}{L-1}, & j \in s_k, \\ \mathcal{F}_y, & j = y, \\ 0, & otherwise. \end{cases} \tag{8}$$

The index subset s_k includes all the indexes except for y that the k-th classifier predicts non-zero probability. Note that the partition S is adaptive concerning the input, which means that for two inputs x_1 and x_2 of the same label y, the partition S_1 and S_2 could be different. For example, for dog image #1 and dog image #2, an individual network may both assign 0.7 probability to label dog, and separately assign 0.1 probability to labels car, frog, ship for image #1 and plane, truck, cat for image #2. This is why the ADP regularizer is named as *adaptive*, and this adaptivity can avoid imposing an artificial bias on the relationship among different classes. When L is large, the optimal solution will still approximately satisfy the conclusion if $K \nmid (L-1)$. As shown in Table 5 of our experiments, even when L is not large ($L = 10$), the ADP method can still work well in the case of $K \nmid (L-1)$. After training by the ADP method, the non-maximal predictions of each network will tend to be mutually orthogonal, and further leads to different feature distributions and decision domains as shown in Fig. 2, where more technical details can be found in Section 4.2.

4. Experiments

We now provide the experimental results to demonstrate the effectiveness of the ADP training on improving adversarial robustness under different attack methods, while maintaining state-of-the-art accuracy on normal examples.

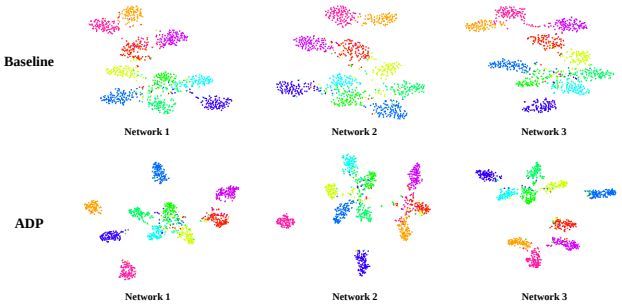

Figure 2. t-SNE visualization results on the final hidden features of each individual network. The input is the test set of CIFAR-10.

4.1. Setup

We choose three widely studied datasets—MNIST, CIFAR-10 and CIFAR-100 (LeCun et al., 1998; Krizhevsky & Hinton, 2009). MNIST consists of grey images of handwritten digits in classes 0 to 9, and CIFAR-10 consists of color images in 10 different classes. CIFAR-100 is similar to CIFAR-10 except that it has 100 classes. Each dataset has 50,000 training images and 10,000 test images. The pixel values of images are scaled to be in the interval $[0, 1]$. The normal examples in our experiments refer to all the ones in the training and test sets. The code is available at https://github.com/P2333/Adaptive-Diversity-Promoting.

4.2. Performance on Normal Examples

We first evaluate the performance in the normal setting. On each dataset, we implement the ensemble model consisting of three Resnet-20 networks (He et al., 2016), where $K = 3$ satisfies the condition $K \mid (L-1)$ for $L = 10$ and $L = 100$. In Section 4.6, we will investigate on the case when $K \nmid (L-1)$. Our baseline training method is to train the ensemble with the ECE loss, which is equivalent to ADP training with $\alpha = \beta = 0$. As analyzed in Section 3.1, simultaneously training all networks with the ECE loss is also equivalent to independently training each one with the corresponding CE loss, which is the most commonly applied strategy on training DNN ensembles (Russakovsky et al., 2015; Goodfellow et al., 2016). About our method, we test two hyperparameter settings: $\mathbf{ADP}_{2,0}$ sets $\alpha = 2$ and $\beta = 0$ corresponding to the case in Theorem 2, where the value of α is chosen according to Eq. (7). This is a reference setting to demonstrate the effect of the ensemble entropy part alone in the ADP regularizer. $\mathbf{ADP}_{2,0.5}$ sets $\alpha = 2$ and $\beta = 0.5$ corresponding to the case in Corollary 1, which is a full version of our method. Later we may omit the subscript as **ADP** to denote this setting without ambiguity.

In Table 1, we show the test error rates on each dataset. For all the three training settings, we use the same training hyperparameters, e.g., learning rate, training epoch, etc. The weight initialization is different for each network. We apply Adam optimizer (Kingma & Ba, 2014) with an initial learning rate of 0.001. Following similar setting as in He et al. (2016), we separately run the training process for 40 epochs on MNIST, 180 epochs on CIFAR-10 and CIFAR-100 with the mini-batch size of 64 on a Tesla P100 GPU worker. As Table 1 shows, although the ADP training causes higher error rates on individual networks, it leads to lower error rates on the ensembles. This verifies that promoting the ensemble diversity as defined in Eq. (3) is also effective to improve ensemble performance in the normal setting (Krogh & Vedelsby, 1995; Tsymbal et al., 2005). To investigate the influence of the ADP training on the feature distributions, we use the t-SNE method (Maaten & Hinton, 2008) to visualize the final hidden features of each network in Fig. 2. We can see that when trained by the baseline method, the three individual networks have learned similar feature distributions. In contrast, when trained by the ADP method, the learned feature distributions are much more different, which makes it more difficult for adversarial examples to transfer among individual networks as demonstrated in Section 4.4.

Table 1. Classification error rates (%) on the test set of each dataset. The ensemble model consists of three Resnet-20 networks.

Dataset	Classifier	Baseline	$\mathrm{ADP}_{2,0}$	$\mathrm{ADP}_{2,0.5}$
MNIST	Net 1	0.44	0.43	0.36
	Net 2	0.39	0.45	0.47
	Net 3	0.43	0.37	0.47
	Ensemble	0.32	0.32	**0.28**
CIFAR-10	Net 1	8.30	9.01	9.34
	Net 2	8.52	9.15	9.34
	Net 3	8.67	9.14	9.92
	Ensemble	6.78	6.74	**6.56**
CIFAR-100	Net 1	35.25	-	39.04
	Net 2	35.91	-	40.86
	Net 3	36.03	-	39.00
	Ensemble	30.35	-	**29.80**

Furthermore, to demonstrate that the ADP method only results in a marginal increase in training time, we test the training time per epoch with the mini-batch size of 32 on CIFAR-10. When $K = 5$, it takes 204s/epoch for the baseline method, and 227s/epoch for the ADP methods; when $K = 20$, it takes 692s/epoch for baseline, and 744s/epoch for ADP. Thus the ADP methods only increase around 10% training time compared to baseline even when $K = 20$.

4.3. Performance under White-box Attacks

In the adversarial setting, there are mainly two threat models (Carlini & Wagner, 2017a): *white-box adversaries* know all the information about the classifier models, including training data, model architectures and parameters; *black-box adversaries* know the classification tasks, but have no access to the information about the classifier models.

In this sub-section, we test the performance of ensemble models defending white-box attacks. We apply the attack methods introduced in Section 2.2. The results are reported in Table 2 and Table 3, where we test each attack on two or three representative values of parameters. The iteration